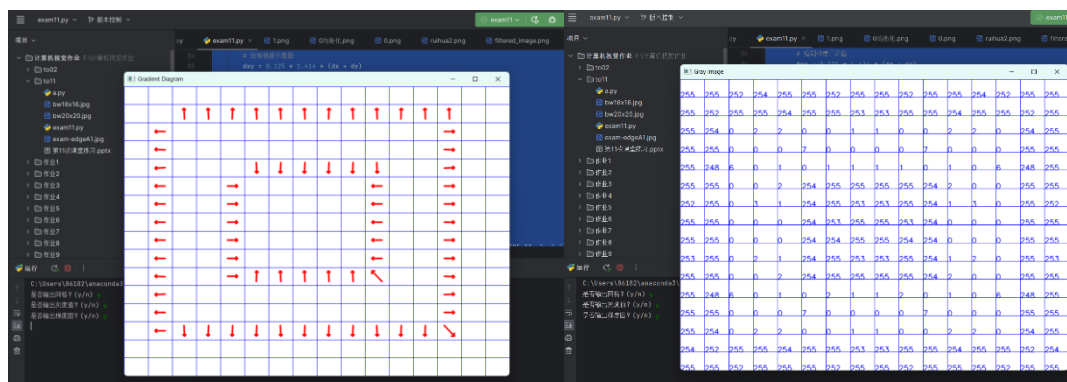


图像处理基础课程课后作业十

建智 2201 谢天赐 20221082044

10.1.1)

程序运行效果如图所示：



可从控制台输出看出：该程序实现了可选择是否输出网格、是否输出灰度值、是否输出梯度图的功能。

代码实现：

```
import cv2
import numpy as np
import math

#定义计算反正切函数（假设角度范围为[-pi, pi]）
def myMath_GetAtan(x, y):
    return math.atan2(y, x)

def process_image(fn):
    #读取灰度图
    src = cv2.imread(fn, 0)
    h, w = src.shape[0], src.shape[1]
    width, height = 800, 600
    #设置初始数组
    grX = np.zeros((h, w), dtype=np.float32)
    grY = np.zeros((h, w), dtype=np.float32)
    grG = np.zeros((h, w), dtype=np.float32)
    grA = np.zeros((h, w), dtype=np.float32)
    #计算 X 方向的导数
    for j in range(w - 1):
        for i in range(h):
            grX[i, j] = int(src[i, j + 1]) - int(src[i, j])
    #计算 Y 方向的导数
    for j in range(w):
        for i in range(h - 1):
            grY[i, j] = int(src[i + 1, j]) - int(src[i, j])
```

```

for i in range(h-1):
    grY[i, j] = int(src[i+1, j]) - int(src[i, j])
# 计算梯度值、方向角, 记录最大梯度
grGmax = 0.0
for i in range(h):
    for j in range(w):
        grG[i, j] = math.sqrt(grX[i, j]**2 + grY[i, j]**2)
        if grG[i, j] > grGmax:
            grGmax = grG[i, j]
        grA[i, j] = myMath_GetAtan(grX[i, j], grY[i, j])
dy = height/h
dx = width/w
# 结果图数组
figV = np.zeros((height, width, 3), dtype=np.uint8) + 255
figXY = np.zeros((height, width, 3), dtype=np.uint8) + 255
figGD = np.zeros((height, width, 3), dtype=np.uint8) + 255
# 询问用户是否输出网格、灰度值和梯度图
output_grid = input("是否输出网格? (y/n)") == 'y'
output_gray = input("是否输出灰度值? (y/n)") == 'y'
output_gradient = input("是否输出梯度图? (y/n)") == 'y'
if output_grid:
    # 绘制网格
    for j in range(w):
        kx = int(dx*j)
        cv2.line(figV, (kx, 0), (kx, height), (255, 0, 0), 1)
        cv2.line(figXY, (kx, 0), (kx, height), (255, 0, 0), 1)
        cv2.line(figGD, (kx, 0), (kx, height), (255, 0, 0), 1)
    for i in range(h):
        ky = int(dy*i)
        cv2.line(figV, (0, ky), (width, ky), (255, 0, 0), 1)
        cv2.line(figXY, (0, ky), (width, ky), (255, 0, 0), 1)
        cv2.line(figGD, (0, ky), (width, ky), (255, 0, 0), 1)
if output_gray:
    # 输出灰度值文本
    myFont = cv2.FONT_HERSHEY_SCRIPT_SIMPLEX
    for i in range(h):
        for j in range(w):
            corB = (255, 0, 0)
            hFont = 16
            fs = 0.5
            cc = str(src[i, j])
            kx = int(j*dx)
            ky = int((i+1)*dy)
            cv2.putText(figV, cc, (kx, ky), myFont, fs, corB, 1, cv2.LINE_AA)

```

```

#显示灰度图
cv2.imshow('GrayImage', figV)
cv2.waitKey(0)
cv2.destroyAllWindows()
if output_gradient:
#绘制梯度示意图
dxy=0.1251.414(dx+dy)
grGMin=0.5grGmax
grScale=dxy/grGmax
corR=(0, 0, 255)
for i in range(h):
for j in range(w):
if grG[i, j]>grGMin:
xc=jdx+0.5dx
yc=(i+1)dy-0.5dy
d=grG[i, j]grScale
a=grA[i, j]
fcos=math.cos(a)
fsin=math.sin(a)
x1=int(xc-dfcos)
y1=int(yc-dfsin)
x2=int(xc+dfcos)
y2=int(yc+dfsin)
cv2.arrowedLine(figGD, (x1, y1), (x2, y2), corR, 2, cv2.LINE_AA, 0, 0.2)
#显示梯度图
cv2.imshow('GradientDiagram', figGD)
cv2.waitKey(0)
cv2.destroyAllWindows()
if __name__=="__main__":
fn="bw16x16.jpg"
process_image(fn)
#设置 matplotlib 的字体为支持中文的字体，例如使用黑体
plt.rcParams['font.sans-serif']=['SimHei']
#读取原始数据
data=[
[1, 2, 9, 12, 13, 12, 14, 14, 15, 15],
[3, 9, 13, 13, 13, 14, 15, 15, 11, 8],
[6, 11, 13, 14, 15, 13, 10, 6, 5, 3],
[10, 14, 13, 14, 11, 3, 1, 0, 0, 0],
[11, 13, 14, 11, 2, 0, 3, 9, 7, 5],
[13, 15, 13, 9, 2, 7, 10, 14, 14, 12],
[13, 12, 11, 2, 5, 10, 14, 14, 13, 14],
[13, 14, 9, 0, 9, 12, 14, 15, 15, 15],
[14, 10, 9, 1, 9, 14, 15, 15, 15, 14],

```

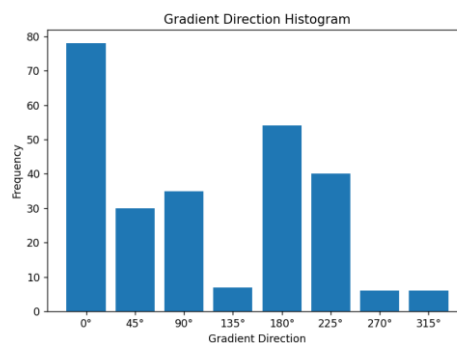
```

[13, 11, 7, 0, 4, 14, 15, 15, 14, 13]
]
#将数据转换为 numpy 数组
image=np.array(data)
#计算原始图像的直方图
hist,bins=np.histogram(image,bins=16,range=(0,16))
#计算累积频率分布
cdf=hist.cumsum()
cdf_normalized=cdf/255/cdf.max()
#映射到新的灰度值
equalized_cdf=np.interp(image.flatten(),bins[:-1],cdf_normalized)
equalized_image=equalized_cdf.reshape(image.shape).astype(np.uint8)
#计算均衡化后的图像的直方图
equalized_hist,equalized_bins=np.histogram(equalized_image,bins=256,range=(0,256))
#绘制原始图像的直方图
plt.figure(figsize=(12,6))
plt.subplot(1,2,1)
plt.title("原始图像的直方图")
plt.xlabel("像素强度")
plt.ylabel("频率")
plt.xlim([0,16])#设置 x 轴的范围
plt.bar(bins[:-1],hist,width=1,color='gray')#绘制直方图
#绘制均衡化后的图像的直方图
plt.subplot(1,2,2)
plt.title("均衡化后图像的直方图")
plt.xlabel("像素强度")
plt.ylabel("频率")
plt.xlim([0,255])#设置 x 轴的范围
plt.bar(equalized_bins[:-1],equalized_hist,width=1,color='gray')#绘制直方图
plt.tight_layout()
plt.show()

```

10.1.2)

程序运行效果直方图如图所示：



```

# 计算梯度方向直方图 (0 - 360度, 45度间隔)
gradient_direction_histogram = np.zeros(8)
for i in range(h):
    for j in range(w):
        # 将梯度方向转换为0 - 360度范围
        gradient_direction_degrees = (grA[i, j] * 180 / np.pi) % 360
        # 量化到0 - 7 (对应0 - 360度, 45度间隔)
        gradient_direction_quantized = int(gradient_direction_degrees / 45)
        gradient_direction_histogram[gradient_direction_quantized] += 1

# 绘制梯度方向直方图
angle_labels = ['0°', '45°', '90°', '135°', '180°', '225°', '270°', '315°']
plt.bar(angle_labels, gradient_direction_histogram)
plt.xlabel('Gradient Direction')
plt.ylabel('Frequency')
plt.title('Gradient Direction Histogram')
plt.show()

```

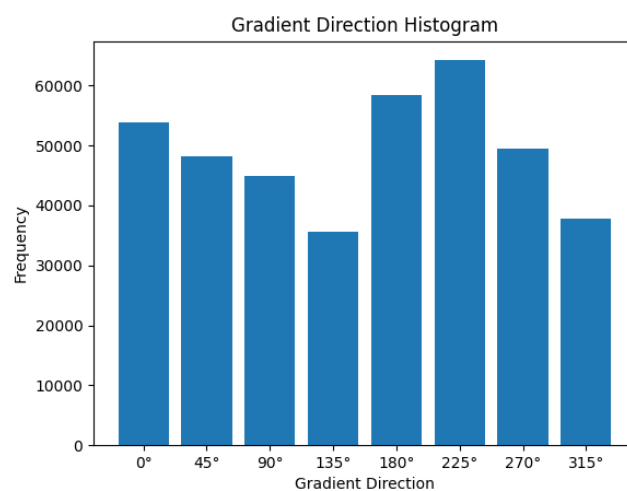
主程序如上图所示。

10.1.3)

程序运行效果直方图如图所示：



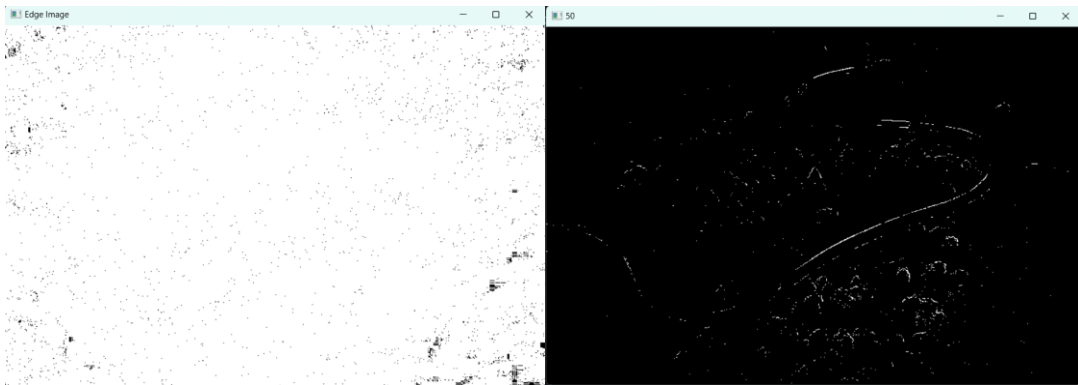
梯度主方向图



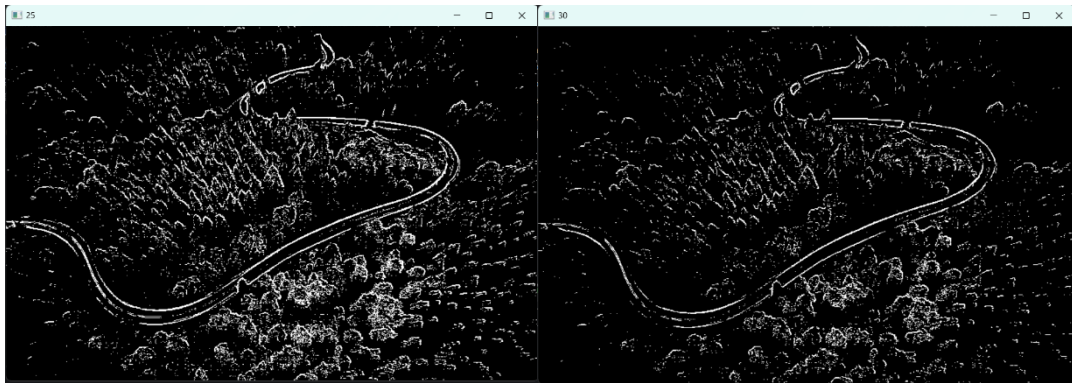
梯度主方向直方图

10.1.4)

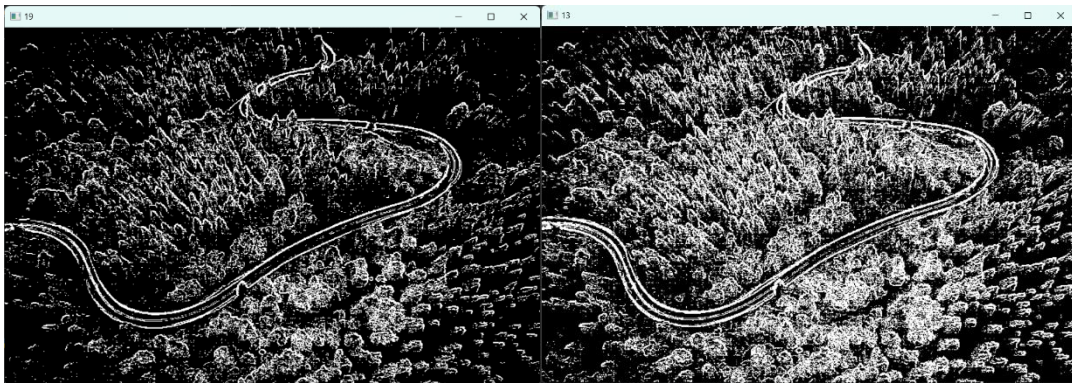
按照梯度阈值编写程序运行的边缘黑白图如图所示：



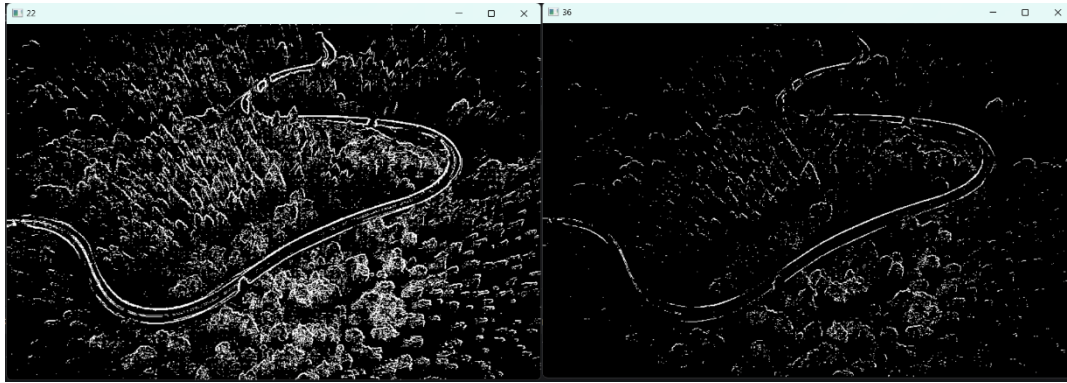
上图为第一次取值分别为 0、50



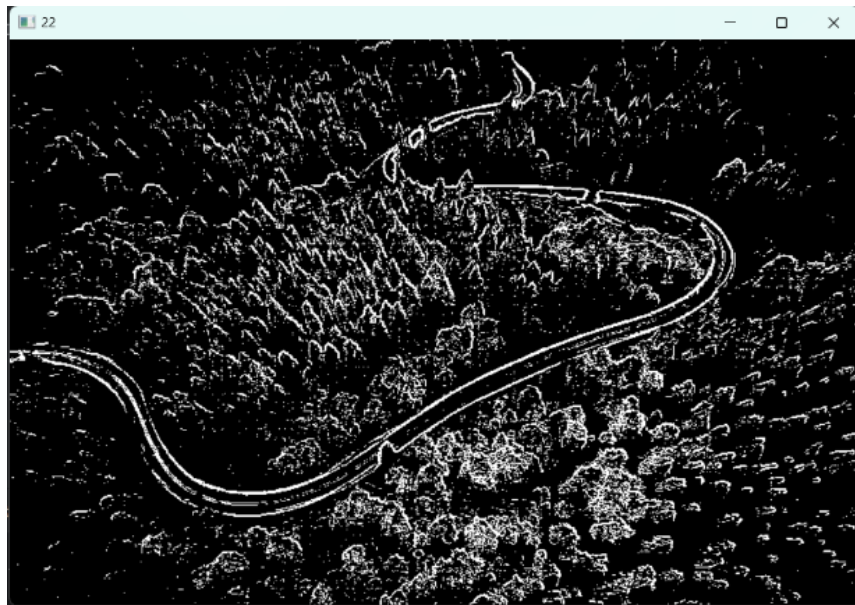
上图为第二次取值分别为 25、30



上图为第三次取值分别为 19、13



上图为第三次取值分别为 22、36
综合考虑图像的清晰度和可读性选择阈值为 22 的图像。



代码实现：

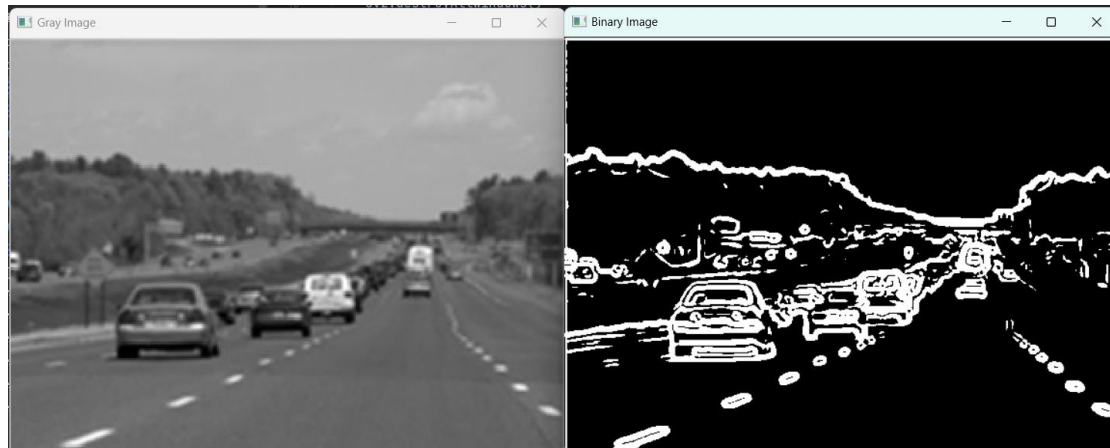
```
import cv2
import numpy as np
import math
import matplotlib
from matplotlib import pyplot as plt
#定义计算反正切函数（假设角度范围为[ -pi, pi]）
def myMath_GetAtan(x, y):
    return math.atan2(y, x)
def process_image(fn):
    #读取灰度图
    src = cv2.imread(fn, 0)
    h, w = src.shape[0], src.shape[1]
    width, height = 800, 600
    #设置初始数组
    grX = np.zeros((h, w), dtype=np.float32)
    grY = np.zeros((h, w), dtype=np.float32)
```

```

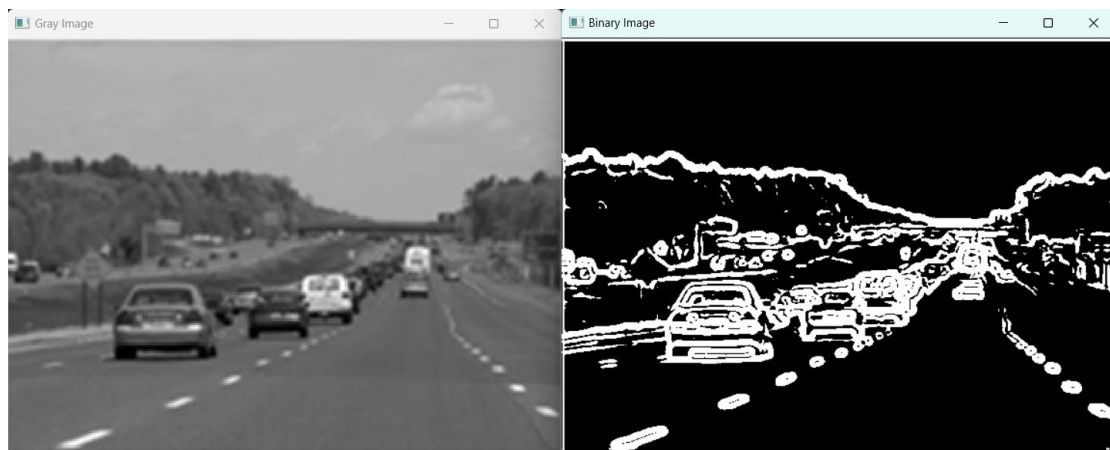
grG=np.zeros((h,w),dtype=np.float32)
grA=np.zeros((h,w),dtype=np.float32)
#计算 X 方向的导数
for j in range(w-1):
    for i in range(h):
        grX[i,j]=int(src[i,j+1])-int(src[i,j])
#计算 Y 方向的导数
for j in range(w):
    for i in range(h-1):
        grY[i,j]=int(src[i+1,j])-int(src[i,j])
#计算梯度值、方向角，记录最大梯度
grGmax=0.0
for i in range(h):
    for j in range(w):
        grG[i,j]=math.sqrt(grX[i,j]**2+grY[i,j]**2)
        if grG[i,j]>grGmax:
            grGmax=grG[i,j]
        grA[i,j]=myMath_GetAtan(grX[i,j],grY[i,j])
dy=height/h
dx=width/w
#结果图数组
figV=np.zeros((height,width,3),dtype=np.uint8)+255
figXY=np.zeros((height,width,3),dtype=np.uint8)+255
figGD=np.zeros((height,width,3),dtype=np.uint8)+255
#计算梯度方向直方图 (0-360 度, 45 度间隔)
gradient_direction_histogram=np.zeros(8)
for i in range(h):
    for j in range(w):
        #将梯度方向转换为 0-360 度范围
        gradient_direction_degrees=(grA[i,j]*180/np.pi)%360
        #量化到 0-7 (对应 0-360 度, 45 度间隔)
        gradient_direction_quantized=int(gradient_direction_degrees/45)
        gradient_direction_histogram[gradient_direction_quantized]+=1
#获取阈值
threshold=int(input("请输入阈值: "))
#根据阈值生成边缘黑白图
edge_image=np.zeros_like(grG,dtype=np.uint8)
edge_image[grG>threshold]=255
cv2.imshow(str(threshold), edge_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
if __name__=="__main__":
    fn="10_1new.jpg"
    process_image(fn)

```

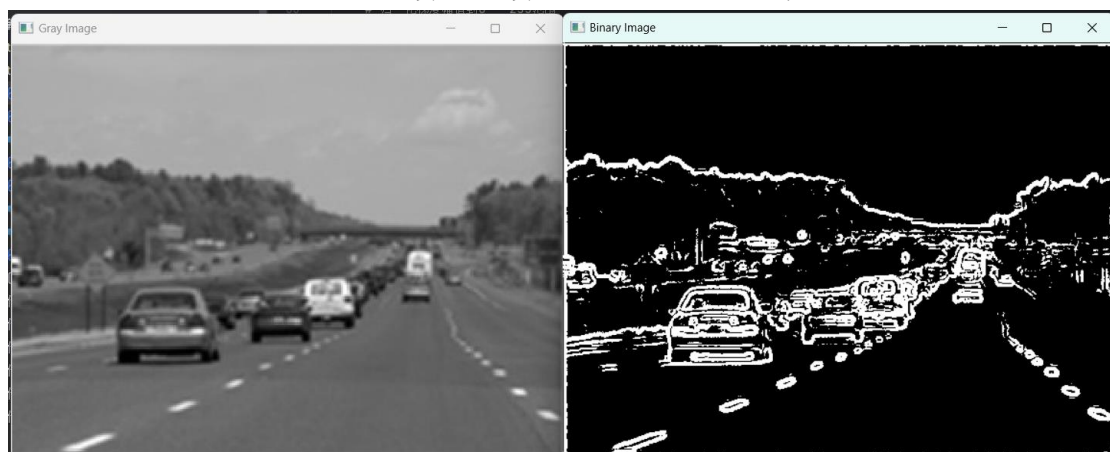

10.2.1)



Sobel 算子计算梯度+自适应阈值确定 (otsu)



Scharr 算子计算梯度+手动调整阈值



Scharr 算子计算梯度+锐化+手动寻找阈值
多方法多次试验对比得出，Scharr 算子计算梯度+锐化+手动寻找阈值基本符合要求，细节完整边缘清晰。

10.2.2)

1. 梯度的基本概念理解

首先，要明白在图像中梯度表示的是像素值变化的程度和方向。想象你在爬

山，山的高度就相当于图像中的像素值。当你从平缓的地方走向陡峭的山坡时，高度变化得很快，这就类似于图像中从颜色比较均匀的区域走到边缘区域时，像素值变化很快。梯度就是用来衡量这种变化的工具。

在图像里，边缘部分的像素值通常会有明显的变化，比如从白色区域过渡到黑色区域。通过计算梯度，我们可以找到这些像素值变化剧烈的地方，也就是边缘的位置。

2. Sobel 算子的方向分解

Sobel 算子把梯度的计算分为水平方向和垂直方向。

水平方向：

它就像是在图像上从左向右看像素值的变化。Sobel 算子通过一个小的滤波器（可以想象成一个小窗口）来查看图像中的一部分像素。这个滤波器在水平方向上有不同的权重。

例如，中间一行的权重比上下两行的权重更大。当这个滤波器在图像上滑动时，它会对经过的像素进行加权求和。如果从左到右像素值增加，那么得到的结果就是一个正值，表示在这个小区域内像素值在水平方向是上升的趋势；如果像素值从左到右减少，结果就是一个负值，表示下降的趋势。

垂直方向：

类似地，垂直方向的 Sobel 算子是从上向下看像素值的变化。它的滤波器也是通过对经过的像素进行加权求和来计算。如果从上到下像素值增加，就得到正值，说明在垂直方向有上升的趋势；如果像素值减少，就得到负值，说明有下降的趋势。

3. 结合水平和垂直方向得到完整梯度信息

单独的水平或垂直方向的梯度只能告诉我们在一个特定方向上像素值的变化情况。但图像中的边缘可能是各种角度的，比如是斜着的。

所以，我们需要把水平和垂直方向的梯度结合起来。通过某种方式（实际上是使用数学公式，比如计算平方和的平方根）来综合这两个方向的信息，得到一个总的梯度幅值。这个幅值表示了在这个像素点周围像素值变化的剧烈程度，幅值越大，说明这个像素点越有可能处于边缘区域。

同时，这个综合后的梯度还可以告诉我们边缘的大致方向。比如，如果水平方向的梯度比垂直方向的梯度大很多，那么边缘可能更倾向于水平方向；反之，如果垂直方向的梯度大，边缘可能更倾向于垂直方向。

简单地计算相邻像素之间的差值来衡量像素值变化可能会受到噪声的干扰。而 Sobel 算子通过对周围多个像素进行加权求和的方式，能够在一定程度上减少噪声的影响。

它的滤波器大小（通常是 3x3）是比较合适的。如果滤波器太小，可能会对噪声过于敏感或者不能很好地捕捉边缘的整体变化；如果滤波器太大，又会模糊边缘的细节，使得边缘位置不准确。Sobel 算子在平衡边缘检测的准确性和对噪声的抗干扰性方面表现较好。

1. Scharr 算子计算梯度的原理

梯度计算的方向分解：

Scharr 算子和 Sobel 算子一样，都是从水平和垂直两个方向来考虑图像像素值的变化。在水平方向上，它像是一个探测器，用于查看图像从左到右的像素变化情况。当它在图像上移动时，会根据所经过像素的亮度变化来判断水平方向

的梯度。如果从左到右像素越来越亮，它会记录下这个变亮的趋势；反之，如果像素越来越暗，就记录下变暗的趋势。

在垂直方向也是类似的操作。它从上到下在图像上滑动，感受像素值的变化，判断是从暗到亮的上升趋势还是从亮到暗的下降趋势。

综合梯度信息：

单独的水平 and 垂直方向的梯度信息是不完整的，因为图像中的边缘可能是各种各样的角度。所以，Scharr 算子会把这两个方向的信息结合起来，得到一个综合的梯度信息。这个综合信息可以告诉我们在某个像素点周围像素值变化的强烈程度。变化越强烈，就越有可能是边缘的位置。而且，通过比较水平和垂直方向的梯度大小，还能大致推测出边缘的方向，比如水平方向梯度大可能意味着边缘更偏向水平走向。

2. Scharr 算子比 Sobel 算子效果好的原因

对边缘细节的捕捉能力更强：

Scharr 算子对像素值变化的敏感度更高。在图像中，一些比较细微的边缘或者是变化不太明显的边缘，Scharr 算子更容易察觉到。就好像用更精细的工具去寻找图像中的边缘线索，它能够发现那些 Sobel 算子可能会遗漏的细微变化。

在复杂纹理区域更具优势：

当图像中有复杂的纹理时，Sobel 算子可能会因为不够敏感而不能很好地分辨出真正的边缘和纹理中的小变化。而 Scharr 算子能够更深入地挖掘这些变化，将真正的边缘从复杂的纹理背景中凸显出来。例如，在一张有细密花纹的布料图像中，Scharr 算子可以更准确地找到布料边缘和花纹中较明显的线条边缘。